

意图路由

Hermes Agent 意图路由 (Intent Routing)

目录

1. 一、总体架构概述
2. 二、命令路由——最核心的意图路由层
 - 2.1 命令注册中心
 - 2.2 命令解析与分发
 - 2.3 ACP Adapter 命令路由
 - 2.4 Skill 命令路由
 - 2.5 Quick Commands
3. 三、模型路由
 - 3.1 概述
 - 3.2 关键词启发式
 - 3.3 路由决策逻辑
 - 3.4 路由集成入口
4. 四、Provider 路由
 - 4.1 OpenRouter Provider 偏好路由
 - 4.2 运行时 Provider 解析
5. 五、消息分发路由
 - 5.1 DeliveryRouter
 - 5.2 交付目标解析规则
6. 六、子代理委托路由
7. 七、平台接入路由
 - 7.1 架构
 - 7.2 平台适配器注册
8. 八、Hook 系统
9. 九、CLI 侧的路由
10. 十、总结
 - 关键设计原则

一、总体架构概述

Hermes Agent 的意图路由并非一个独立的模块或 ML 驱动的意图识别系统，而是一套多层次、多路复用（multiplex）的分层决策机制。它在用户消息进入系统后，在不同层级上完成路由决策：

层次	路由目标	驱动方式	所在文件
命令路由	/ 开头的内置命令 vs Skill 命令 vs Quick Commands 字符串前缀匹配 + 命令注册表		gateway/run.py、acp_adapter/server.py
模型路由	选择强模型（默认）还是便宜模型	关键词启发式规则	agent/smart_model_routing.py
消息路由	消息分发到哪个 Messaging Platform Adapter	Session Source 追踪	gateway/delivery.py
工具路由	LLM 决策调用哪个工具	LLM 自身决策	tools/registry.py
子代理路由	委托任务分配给哪个子 agent/provider	配置驱动	tools/delegate_tool.py
平台路由	处理事件来自哪个消息平台	Event Source 解析	gateway/run.py

二、命令路由 (Command Routing) ——最核心的意图路由层

2.1 命令注册中心 (Central Registry)

文件: hermes_cli/commands.py

这是整个命令路由系统的“单一真相来源”（Single Source of Truth）。

```
# 数据结构
@dataclass(frozen=True)
class CommandDef:
    name: str # 规范名称, 无斜杠: "background"
    description: str # 人类可读描述
    category: str # 分类: "Session", "Configuration"等
    aliases: tuple[str, ...] = () # 别名: ("bg",)
    args_hint: str = "" # 参数提示: "<prompt>", "[name]"
    subcommands: tuple[str, ...] = () # Tab补全的子命令
    cli_only: bool = False # 仅在 CLI 可用
    gateway_only: bool = False # 仅在 Gateway/消息平台可用
    gateway_config_gate: str | None = None # 配置门控
```



所有命令以 CommandDef 条目注册到 COMMAND_REGISTRY 列表。关键内置命令分类：

Session 类:

- /new, /reset — 新会话
- /retry — 重试上一条消息
- /undo — 移除最后一次交换
- /compress — 手动压缩对话上下文
- /background (/bg) — 后台异步运行提示词
- /btw — 临时侧问 (ephemeral side question, 不使用工具、不持久化)
- /queue (/q) — 排队一条消息 (不中断当前运行)
- /rollback — 恢复文件系统检查点
- /stop — 终止所有后台进程
- /approve, /deny — 审批/拒绝危险命令

Configuration 类:

- /reasoning — 管理推理 effort 和显示
- /verbose — 切换工具进度显示模式
- /yolo — 跳过所有危险命令审批
- /personality — 设置预定义人格
- /provider — 显示当前 provider

Tools & Skills 类:

- /skills — 搜索/安装/查看技能
- /cron — 管理定时任务
- /reload-mcp — 重新加载 MCP 服务器
- /browser — 连接浏览器工具
- /plugins — 列出已安装插件

Info 类:

- /commands — 分页浏览所有命令
- /help — 显示帮助
- /usage — 显示 token 使用情况
- /insights — 使用洞察分析
- /platforms — Gateway 平台状态

2.2 命令解析与分发 (Gateway 层)

文件: gateway/run.py — _handle_message() 方法 (约 1678 行)

这是 Gateway 接收来自各消息平台 (telegram、discord、slack 等) 事件的入口方法。消息处理流水线如下:

接收 MessageEvent

- ↓
1. 用户授权检查 (_is_user_authorized)
↓
2. 运行中 Agent 冲突检查
 - ├ 若有 agent 在运行 → 特殊命令 (/stop, /new, /queue) 特殊处理
 - ├ /stop → 硬终止
 - ├ /new → 中断 + 重置会话
 - └ /queue → 追加到队列, 不中断
 - └ 无运行中 agent → 继续
↓
3. 提取命令: event.get_command()
↓
4. 命令别名解析: resolve_command() → CommandDef.name
↓
5. 逐个 if 分支匹配 canonical 命令名
 - ├ 内置命令 → 对应 _handle_*_command() 方法
 - ├ Skill 命令 → build_skill_invocation_message()
 - ├ Quick Commands (user-defined) → 直接执行
 - └ 插件命令 → get_plugin_command_handler()
↓
6. 所有命令均不匹配 → 将消息作为普通用户输入发给 LLM



2.2.1 MessageEvent.get_command() — 命令提取

文件: gateway/platforms/base.py — MessageEvent 类

```
class MessageEvent:
    text: str
    message_type: MessageType = MessageType.TEXT
    source: SessionSource = None

    def is_command(self) -> bool:
        return self.text.startswith("/")

    def get_command(self) -> Optional[str]:
        """从 `/` 开头的消息中提取命令名。"""
        if not self.is_command():
            return None
        parts = self.text.split(maxsplit=1)
        raw = parts[0][1:].lower() if parts else None
        if raw and "@" in raw:
            raw = raw.split("@", 1)[0] # 处理 telegram @mention 格式
        return raw

    def get_command_args(self) -> str:
        """获取命令后的参数。"""
        if not self.is_command():
            return self.text
        parts = self.text.split(maxsplit=1)
        return parts[1] if len(parts) > 1 else ""
```



路由决策点: 这是一个纯字符串操作, 通过检查 text.startswith("/") 判断是否为命令。无需任何 ML 或复杂 NLP。

Note

命令路由本质是一个确定性有限自动机 (DFA)

负路由 (Negative Routing) 模式: Hermes 的命令分发隐含了一个与大多数意图分类系统相反的设计原则: **只有“未识别”的命令才流向 LLM**。这叫负路由 (或者说“LLM 作为最终兜底”)

2.2.2 命令别名解析

文件: hermes_cli/commands.py — resolve_command() 函数

```
def resolve_command(name: str) -> CommandDef | None:
    """将命令名或别名解析为 CommandDef。"""
    return _COMMAND_LOOKUP.get(name.lower().lstrip("/"))
```

```
# 构建查找表
_COMMAND_LOOKUP: dict[str, CommandDef] = _build_command_lookup()
# 其中包含命令名和所有别名 (如 "bg" → CommandDef("background", ...))
```



关键: gateway/run.py 中在分发前会将别名解析为规范名:

```
_cmd_def = _resolve_cmd(command) if command else None
canonical = _cmd_def.name if _cmd_def else command
# 后续所有 if 判断都基于 canonical 名称
```



2.2.3 GATEWAY_KNOWN_COMMANDS — Gateway 可识别命令集合

```
GATEWAY_KNOWN_COMMANDS: frozenset[str] = frozenset(
    name
    for cmd in COMMAND_REGISTRY
    if not cmd.cli_only or cmd.gateway_config_gate # 包含所有 gateway 端命令
    for name in (cmd.name, *cmd.aliases)
)
```



用途:

- 1. **Telegram BotCommands 菜单** — Telegram 只支持 /command 格式菜单
- 2. **Slack 子命令映射** — slack_subcommand_map() 生成 slash handler 映射
- 3. **命令补全** — SlashCommandCompleter 提供 tab 补全

2.3 ACP Adapter 命令路由 (Headless 场景)

文件: acp_adapter/server.py

ACP (Agent Client Protocol) 适配器在 headless 场景下处理来自编辑器 (如 Claude Code) 的消息:

```
class HermesACPAgent(acp.Agent):
    # 截取 slash 命令 (无需 LLM 调用)
    if user_text.startswith("/"):
        response_text = self._handle_slash_command(user_text, state)
        if response_text is not None:
            # 直接返回文本响应, 不走 LLM
            return PromptResponse(stop_reason="end_turn")

    # 未识别命令 → 发给 LLM 处理 (用户可能把 /something 当作文本输入)
```



内置的 headless 命令列表:

```
_SLASH_COMMANDS = {
    "help": "Show available commands",
    "model": "Show or change current model",
    "tools": "List available tools",
    "context": "Show conversation context info",
    "reset": "Clear conversation history",
    "compact": "Compress conversation context",
    "version": "Show Hermes version",
}
```



Note

Headless 场景就是“没有图形界面、没有音频设备、没有交互式终端”的服务器环境, Hermes Agent 通过环境检测、懒加载、safe wrapper等手段优雅降级, 确保在这种情况下仍能正常运行而不是崩溃

2.4 Skill 命令路由 (动态命令)

文件: agent/skill_commands.py — scan_skill_commands()

Skills 以 SKILL.md 文件形式存储在 ~/.hermes/skills/ 目录, 每个 skill 会自动注册为一个 slash 命令:

```
def scan_skill_commands() -> Dict[str, Dict[str, Any]]:
    """扫描 skills/ 目录, 返回 /command-name → skill 信息的映射。"""
    # 遍历 ~/.hermes/skills/**/*.md
    # 解析 frontmatter 中的 name 和 description
    # 命令名 = name.lower().replace(' ', '-')
    # 注册到 _skill_commands 全局字典
```



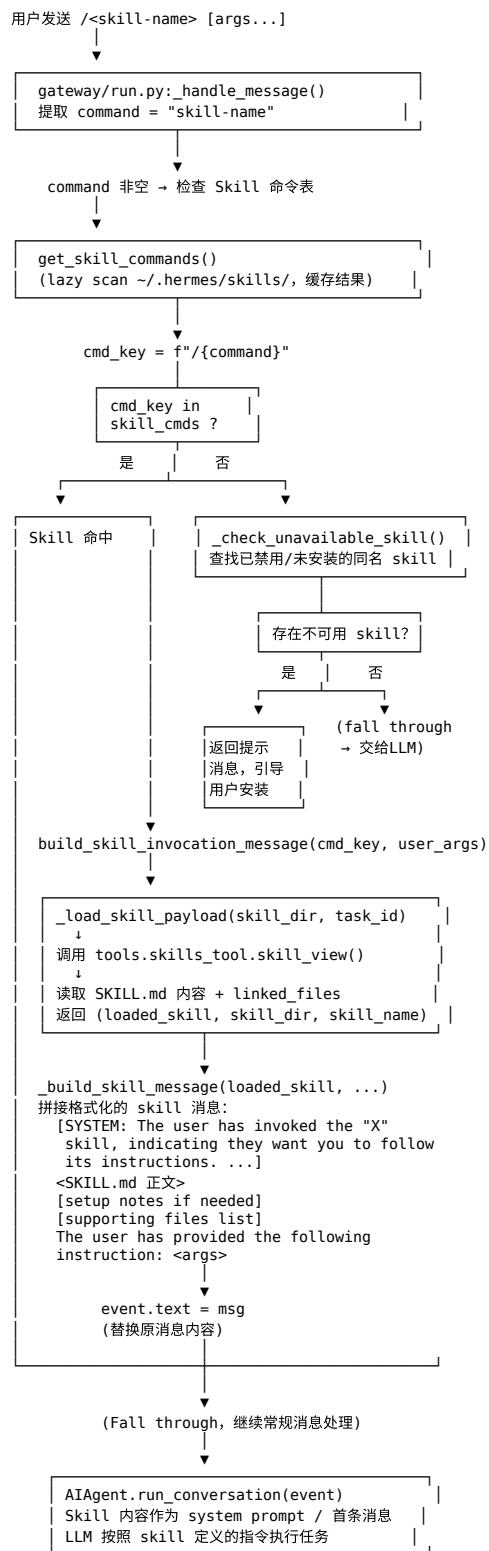
Skill 命令分发流程 (gateway/run.py 第 2028-2048 行):

Skill 以 SKILL.md 文件形式存储在 ~/.hermes/skills/ 目录, 系统启动时由 scan_skill_commands() 扫描注册. 命令名由 frontmatter 中的 name 字段派生 (空格替换为 -, 转小写)。

```
if command:
    from agent.skill_commands import get_skill_commands, build_skill_invocation_message
    skill_cmds = get_skill_commands() # 返回 {"skill-name": {...}, ...}
    cmd_key = f"/{command}"
    if cmd_key in skill_cmds: # Skill 命令命中
        user_instruction = event.get_command_args().strip()
        msg = build_skill_invocation_message(cmd_key, user_instruction, task_id=quick_key)
        if msg:
            event.text = msg # 替换消息内容为 skill prompt
            # 继续 fall through → 交给 LLM 处理 (skill 内容作为 prompt)
    else:
        # 尝试匹配"已知但已禁用/未安装"的 skill → 给出可操作的提示
        _unavail_msg = _check_unavailable_skill(command)
        if _unavail_msg:
            return _unavail_msg
```



以下是完整的 Skill 命令分发流程图:



2.5 Quick Commands (用户自定义快速命令)

文件: gateway/run.py 第 1971-2010 行

用户可在 config.yaml 中定义无需 LLM 调用的快速命令:

```
quick_commands:
  hello:
    type: exec
    command: "echo Hello from Hermes!"
  greet:
    type: alias
    target: /background
```



- type: exec → 直接执行 shell 命令, 返回输出
- type: alias → 别名为另一个命令, 继续分发流程

三、模型路由（Model Routing）—— Smart Cheap vs. Strong

3.1 概述

它根据用户输入的**表层特征**（长度、关键词等），在单轮对话级别决定使用**便宜模型**还是**默认强模型**。

文件: agent/smart_model_routing.py

Note

每轮独立决策 vs 跨轮状态——放弃 MDP 建模的动机

choose_cheap_model_route() **对每条消息独立做决策** ("stateless per-turn")，没有引入跨轮对话的隐状态（如"上一轮走了便宜模型且用户没有不满"→继续走便宜）。这是一个有意放弃更强模型（Markov Decision Process / Bandit）的选择。

从算法视角看，跨轮状态建模可以捕获用户的"隐式反馈"（如用户是否立即追问/抱怨）来动态调整路由，但代价是：（1）实现复杂度大幅提升；（2）需要额外的埋点和反馈收集机制；（3）状态空间膨胀后需要在线学习或规则引擎维护。

当前独立决策模型等价于假设每轮消息的复杂度与其他轮无关——这当然是过度简化，但在实践中，只要大多数用户的简单问题确实能在单轮内解决，这个简化带来的误差就在可接受范围内。

3.2 关键词启发式（Simple Intent Detection）

```
_COMPLEX_KEYWORDS = {
    "debug", "debugging", "implement", "refactor", "patch",
    "traceback", "stacktrace", "exception", "error",
    "analyze", "analysis", "investigate",
    "architecture", "design", "compare", "benchmark", "optimize",
    "review", "terminal", "shell", "tool", "tools",
    "pytest", "test", "tests",
    "plan", "planning", "delegate", "subagent",
    "cron", "docker", "kubernetes",
}
```



命中任一词条即触发强模型；无命中且满足长度约束则走便宜模型。此集合为人工维护，完全可解释，但覆盖范围受限于维护者经验（如含 "debug" 不含 "troubleshoot"）。

3.3 路由决策逻辑

```
def choose_cheap_model_route(user_message: str, routing_config) -> Optional[Dict]:
    """若消息看起来简单，返回配置的便宜模型；否则返回 None（使用主模型）。"""

    # 决策条件（全部满足才走便宜模型）：
    if len(text) > max_chars (默认160): return None
    if word_count > max_words (默认28): return None
    if newline_count > 1: return None
    if ```` in text or `` in text: return None # 代码
    if URL_RE.search(text): return None # URL
    if words & _COMPLEX_KEYWORDS: return None # 复杂关键词

    # 以上全部不触发 -> 使用便宜模型
    return cheap_model_config
```



特点：

- **保守策略**：宁可误用强模型，也不愿在复杂任务上用便宜模型失败
- **零-shot**：完全基于手工规则，无任何训练
- **可配置**：config.yaml 中配置 cheap_model 和阈值参数

Note

words & _COMPLEX_KEYWORDS —— 集合交集作为多特征分类器

算法等价于 0-1 权重的线性分类器（命中 = 1，阈值 = 1）：

- **优势**：完全可解释——检查两个集合即可理解路由结果，无需日志
- **代价**：词形脆弱——"analyze" 在集合中，"analysis" 不在，一词多形会漏判

Note

保守策略的本质是 Precision-Recall 权衡的极端选择

两种错误的代价严重不对称：

- **漏判（复杂→便宜）**：任务失败、工具调用错误、需用户重试 → 代价高
- **误判（简单→强模型）**：多花费用 → 代价低

因此阈值卡得很紧（max_chars=160、max_words=28、含代码一律拒绝），等价于 Bayesian decision theory 中损失不对称时的最优保守边界。

Note

关键词集的幸存者偏差陷阱

集合中的词 = "被观察到出现在复杂任务中"的词；不在其中的词可能：

1. 确实与简单任务强相关
2. 也出现在复杂任务中但未被取样到
3. 歧义词（如 "test"：单元测试请求是复杂的，日常对话是简单的）

建议：定期跑离线分析，统计 P(simple|keyword)，用 Precision/Recall 曲线评估增删词汇。

3.4 路由集成入口

文件: gateway/run.py 第 779-791 行

```
def _resolve_turn_agent_config(self, user_message: str, model: str, runtime_kwargs: dict) -> dict:
    from agent.smart_model_routing import resolve_turn_route
    primary = {"model": model, ...}
```

```
return resolve_turn_route(user_message, self._smart_model_routing, primary)
```



每轮消息都会调用一次，进行独立的路由决策。

四、Provider 路由（Provider Routing）

4.1 OpenRouter Provider 偏好路由

文件: gateway/run.py 第 996-1007 行, hermes_cli/runtime_provider.py

从 config.yaml 加载:

```
provider_routing:
  # 为特定模型指定偏好的 OpenRouter provider
  # 例如强制某些模型走指定 provider 以获得更好价格/可用性
```



4.2 运行时 Provider 解析

文件: hermes_cli/runtime_provider.py — resolve_runtime_provider()

支持多 provider 的统一解析:

- openrouter — OpenRouter 聚合平台
- nous — Nous Research
- zai — Zai 平台
- kimi-coding — Kimi Coding
- minimax — MiniMax
- openai — OpenAI Direct
- custom — 用户自定义 provider

五、消息分发路由（Delivery Routing）

5.1 DeliveryRouter

文件: gateway/delivery.py — DeliveryRouter 类

Cron 任务输出和 Agent 响应的分发目标解析:

```
@dataclass
class DeliveryTarget:
    platform: Platform          # telegram, discord, slack, feishu, ...
    chat_id: Optional[str] = None # None → 使用 home channel
    thread_id: Optional[str] = None
    is_origin: bool = False      # 是否回源
    is_explicit: bool = False     # 是否显式指定

    @classmethod
    def parse(cls, target: str, origin: Optional[SessionSource]) -> "DeliveryTarget":
        """解析交付目标字符串。"""
        # "origin" → 回源
        # "local" → 本地文件
        # "telegram" → Telegram home channel
        # "telegram:123456789" → 指定 chat
```



Note

多目标去重的三重 key —— 业务不变性 vs 实现简洁性的权衡

去重 key = (platform, chat_id, thread_id) 三元组, is_origin 未纳入 key:

- ["telegram", "origin"] 且 origin = telegram → 去重只发一次 ✓
- ["telegram", "origin"] 且 origin = discord → 两条都保留 ✓

"origin" 字符串在 to_string() 中被归一化为 origin.platform, 与显式 platform 等价——这是一种**语义归一化**: 不同字符串表示同一目标时, 在投递前统一化, 无需因来源标记不同而重复发送。

Note

逐跳追加 vs 整体去重 —— 写入顺序依赖风险

always_log_local=True + deliver 中已有 "local" → local 被追加两次 (循环中一次, 末尾一次)。实现通过去重 key 防止了重复投递, 但两次追加逻辑本身易引发困惑。

建议: 用 seen_platforms 在循环内统一处理 always_log_local, 消除末尾追加的隐式行为。

Note

平台消息截断是防御性编程, 而非路由决策

MAX_PLATFORM_OUTPUT = 4000 是对下游平台 API 约束的主动裁剪:

- **定位:** DeliveryRouter 职责是“送到哪”, 不是“能送多大”
- **策略:** 不同平台限速不同, 运行时才能精确获知, 防御性裁剪是最稳健的降级
- **保障:** 截断处留指针 (full output saved to {path}), 消息内容不丢失

与“保守策略” (宁可不用便宜模型) 一脉相承: 宁可丢失尾部, 也不愿消息整体投递失败。

5.2 交付目标解析规则

```
# 支持的 target 格式:
# "origin"      → 转发到请求来源的消息平台
# "local"       → 仅保存到本地文件
# "telegram"    → Telegram home channel
# "telegram:123456" → 指定 Telegram chat
```

["local", "telegram"] → 多目标



Cron Job 的 deliver 字段使用此机制。

六、子代理委托路由（Delegation Routing）

文件: tools/delegate_tool.py

当 Agent 使用 delegate_task 工具时：

```
def resolve_delegation_provider(configured_provider: str, ...) -> Dict:
    """解析子代理使用的 provider 和凭证。"""
    # 支持为子代理配置不同的 provider
    # 例如：父 agent 用 Nous，子代理用 OpenRouter + 便宜模型
```



config.yaml 中的 delegation 段落可配置：

- 子代理使用的 provider
- 子代理使用的模型
- 子代理可用的工具集（toolsets）

七、平台接入路由（Platform Adapter Routing）

7.1 架构

文件: gateway/platforms/ 目录

每条消息通过 SessionSource 标识来源平台：

```
@dataclass
class SessionSource:
    platform: Platform      # telegram, discord, slack, feishu, ...
    chat_id: str
    user_id: str
    thread_id: Optional[str] = None
    chat_type: str = "dm"   # "dm" | "group"
```



7.2 平台适配器注册

文件: gateway/run.py

```
class GatewayRunner:
    adapters: Dict[Platform, BasePlatformAdapter] = {}

    # 启动时注册所有启用的平台适配器
    # 运行中动态增删（热插拔）
    # 消息 → _handle_message() → 检查 source.platform → 对应 adapter
```



八、Hook 系统——可扩展的事件路由

文件: gateway/hooks.py — HookRegistry 类

事件驱动的可扩展路由：

```
# 支持的事件类型：
gateway:startup      # Gateway 进程启动
session:start        # 新会话创建（首次消息）
session:end          # 会话结束 (/new 或 /reset)
session:reset        # 会话重置完成
agent:start          # Agent 开始处理消息
agent:step           # 工具调用循环的每轮
agent:end            # Agent 处理完成
command:*            # 任意 slash 命令执行（通配符）
```



Wildcard 匹配： handler 可注册 "command:*" 来监听所有命令事件。

Note

通配符路由的本质是前缀匹配，可升级为 Trie

"command:*" 在算法上等价于字符串前缀匹配，对应 Trie 或 AC 自动机的经典场景：

- 当前实现：** 遍历所有已注册 pattern 做前缀比对
- 规模瓶颈：** hook 数量达到数百时，O(n) 遍历成为瓶颈
- 升级路径：** 基于 Trie 的前缀索引可将匹配降为 O(m)（m = 命令名长度）

当前规模（几个到几十个 hook）下遍历足够，属于“够用但预留扩展性”的设计边界。

九、CLI 侧的路由

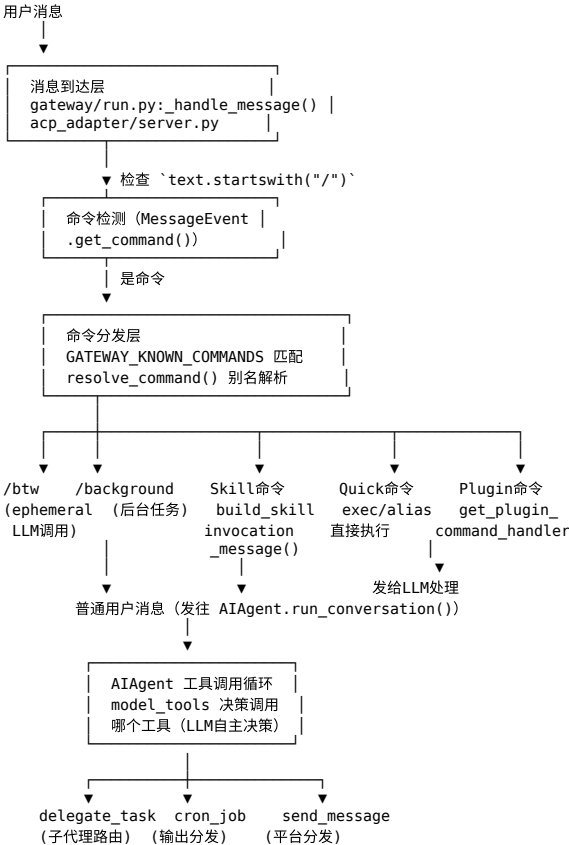
文件: cli.py（主 CLI REPL）

CLI 侧的路由较为简单，通过 prompt_toolkit 的 SlashCommandCompleter 提供命令补全：

```
# CLI 输入 → 以 `/` 开头 → 走命令路由
# 否则 → 正常发给 LLM
```



十、总结：Hermes 意图路由全景图



关键设计原则

1. **命令路由优先于 LLM**: 所有以 / 开头的消息先走命令路由, 未命中才发给 LLM
2. **别名统一解析**: 所有别名在分发前被规范化为 canonical name
3. **分层决策**: 模型路由在消息层, 交付路由在输出层, 平台路由在接入层
4. **无 ML 意图识别**: 所有路由决策基于字符串匹配或关键词规则, 无训练模型
5. **可扩展性**: 通过 Hook 系统、register_plugin_command() 和 Skill 机制支持动态扩展
6. **零歧义处理**: 别名只指向一个 canonical 命令, 不存在一词多义

Note

Hermes 路由的隐含哲学: 确定性优于灵活性, 延迟优于能力

综合审视整份报告, 一个贯穿全系统的算法哲学浮现出来: Hermes 的路由系统本质上是一个**规则优先 (rule-first)**、**LLM兜底 (LLM-as-fallback)** 的分层决策树, 而非当前业界流行的“LLM作为大脑统一调度一切”的方案。这在算法上有几个深层含义:

- **零幻觉路由**: 规则路由永远不会“想错”——给定相同输入, 保证相同输出。LLM 路由 (如果有的话) 天然带有非确定性, 同一输入可能因 temperature 或上下文而得到不同路由决策。在任务关键系统里, 这是一个不可忽视的可靠性差异。
- **延迟 budget 的主动管理**: 每增加一层 LLM 调用, p99 延迟增加 100-500ms。规则路由的典型延迟在微秒级 (dict lookup) 到毫秒级 (I/O), 差距在 2-3 个数量级。在多租户 Gateway 场景下, 规则优先直接决定了系统的容量上限。
- **成本可预测性**: LLM 调用有 token 成本, 且随流量非线性增长 (长对话上下文膨胀)。规则路由的成本是固定的基础设施成本, 更容易做容量规划和预算控制。

这套设计在当前固定命令集 (数百个 slash 命令) + 固定平台集 (十余个消息平台) 的约束下是合理且高效的。但如果未来需要支持**用户自定义的自然语言意图** (如“帮我分析一下最近的日志”), 这套系统将面临根本性的扩展挑战: 关键词规则会组合爆炸, O(n) 规则维护不可持续, 届时引入轻量级嵌入模型 (embedding-based classifier) 做自然语言意图匹配几乎是必然路径

Note

与业界主流方案的对比: 规则系统 vs LLM 路由的取舍

核心洞察: 选择什么路由策略, 取决于输入空间是封闭的还是开放的。

输入空间	最优方案	原因
封闭 (slash 命令有限可枚举)	规则路由	可证明正确性 + O(1) 复杂度
开放 (自然语言意图)	LLM 路由	无法穷举, 只能语义理解

Hermes 的 slash 命令空间是封闭的, 设计者做出了正确的技术选型。LLM 原生路由 (LangChain Agent Router、OpenAI Function Calling) 适用于开放域场景, 但在封闭空间里是“杀鸡用牛刀”。